

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №10
з курсу «Хмарні технології та великі дані»
на тему:
«Запустити перший Docker-контейнер»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	4
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	6
Висновок	10
Код програми	11

Тема

Базовий життєвий цикл контейнеризованих застосунків: встановлення локального середовища виконання контейнерів, завантаження публічних образів та оркестрація першого Docker-контейнера.

Мета

Дослідження фундаментальних засад технології контейнеризації на рівні операційної системи та практичне освоєння базового робочого процесу Docker. Розгортання та конфігурація локального середовища виконання на базі платформи Rancher Desktop з інтеграцією бекенду Moby (dockerd). Отримання практичних навичок завантаження готових образів із публічного реєстру Docker Hub, конфігурації мапінгу мережевих портів хостової системи на порти контейнера, ідентифікації сутностей за власними іменами, моніторингу системних журналів, а також управління життєвим циклом (зупинка та утилізація) ізолюваних середовищ через Docker CLI.

Вхідні дані

Параметр / Інструмент	Значення / Тип	Призначення
Середовище віртуалізації	Rancher Desktop (WSL2 Engine)	Локальна платформа з відкритим кодом для запуску рушія контейнеризації без сторонніх ліцензійних обмежень.
Рушій контейнерів	dockerd (Moby)	Демон контейнеризації, що надає стандартний Docker API для керування процесами через утиліту командного рядка.
Базовий образ	nginxdemos/nginx (latest)	Легковаговий публічний образ веб-сервера Nginx, призначений для демонстрації обробки трафіку та діагностики контейнера.
Мережевий порт хоста	8080 (TCP)	Порт на локальній машині розробника, на який спрямовується вхідний трафік.
Мережевий порт контейнера	80 (TCP)	Внутрішній стандартний порт веб-сервера усередині ізолюваного неймспейсу контейнера.
Мапінг портів	8080:80 (-p)	Механізм Network Address Translation (NAT), що зв'язує сокет хостової операційної системи із сокетом процесу в контейнері.
Ідентифікатор контейнера	my-web-app (--name)	Статичне символічне ім'я, призначене для явної адресації контейнера в командах моніторингу та управління.

Завдання

- розгорнути та налаштувати локальне середовище контейнеризації на базі Rancher Desktop, обравши рушій dockerd (moby) для збереження нативної підтримки команд стандарту Docker CLI.
- здійснити завантаження скомпільованого образу веб-застосунку nginxdemos/hello з офіційного публічного реєстру Docker Hub та верифікувати його наявність у локальному сховищі образів.
- ініціалізувати й запустити контейнер у фоновому автономному режимі, задавши унікальне символічне ім'я та налаштувавши переадресацію трафіку з локального порту 8080 на внутрішній порт 80.
- провести функціональне тестування працездатності розгорнутого веб-сервера шляхом відправлення HTTP-запитів через консольну утиліту curl та верифікації повернення статус-коду 200 OK.
- дослідити поведінку застосунку за допомогою вичитування системних журналів (логів) доступу та помилок стандартного потоку виведення stdout/stderr.
- ознайомитися з процедурою коректного виведення інстансу з експлуатації через надсилання сигналу SIGTERM (зупинка) та повної утилізації ресурсів контейнера (видалення).

Критерії виконання роботи

- *Ізоляція та цілісність*: запуск контейнера виконується у фоновому detached-режимі без блокування керуючого термінала, із коректною ізоляцією процесу на рівні системних просторів імен (namespaces) та контрольних груп (cgroups).
- *Коректність трансляції мережі*: застосунок усередині контейнера повинен безперешкодно приймати вхідні TCP-з'єднання на локальній петлі хоста localhost:8080, успішно транслюючи їх на внутрішній веб-сервер.
- *Детермінізм управління сутностями*: керування контейнером здійснюється виключно за присвоєним аліасом my-web-app без необхідності використання випадково згенерованих хеш-ідентифікаторів.
- *Повнота життєвого циклу*: демонстрація всіх стадій стану контейнера (Created -> Running -> Stopped -> Removed) із відповідним підтвердженням змінами у виводах діагностичних команд.

Теоретичні відомості

Технологія контейнеризації становить сучасний стандарт пакування, доставки та запуску програмного забезпечення. На відміну від класичної повної віртуалізації (гіпервізорів типу Type-1 або Type-2), яка вимагає емуляції апаратного забезпечення та розгортання окремого гостьового ядра операційної системи, контейнеризація реалізує ізоляцію безпосередньо на рівні спільного ядра ОС хоста. Основними примітивами ядра Linux, що забезпечують таку ізоляцію, є простори імен (namespaces — для розмежування ідентифікаторів

процесів PID, мережевих інтерфейсів NET, точок монтування MNT та міжпроцесорної взаємодії IPC) та контрольні групи (cgroups — для жорсткого лімітування виділеної оперативної пам'яті, ресурсів CPU та дискового вводу-виводу).

Образ контейнера (Container Image) є незмінним (immutable) шаблоном, що складається із набору послідовних шарів (layers), сформованих за принципом копіювання при записі (Copy-on-Write, CoW). Такий підхід забезпечує максимальну повторюваність збірок, прискорює кешування та суттєво мінімізує обсяг займаного дискового простору. Контейнер є екземпляром виконання образу, що отримує власний тонкий шар для запису (Writable Container Layer).

Утиліта Rancher Desktop виступає сучасною відкритою альтернативою платформі Docker Desktop, надаючи готову віртуальну машину на базі WSL2 або QEMU/Lima з інтегрованими сервісами управління контейнерами. Використання рушія dockerd (moby) дає змогу взаємодіяти із середовищем через традиційний клієнт Docker CLI, оперуючи сокетом контейнеризатора за допомогою стандартних викликів REST API.

Структура програмної реалізації

Архітектура лабораторного стенду базується на хостовій системі, де через інтерфейс командного рядка відправляються керуючі інструкції до демона Moby всередині ізольованої підсистеми Rancher Desktop.

У межах дослідження виконується наступний лінійний ланцюжок процедур:

- завантаження маніфесту та бінарних шарів тестового веб-сервера через клієнтську команду `docker pull nginxdemos/hello`.
- валідація локального реєстру образів та їхніх метаданих (хеш SHA256, тег, розмір) за допомогою `docker images`.
- створення нового простору імен та запуск ізольованого процесу із прокиданням сокетів `docker run -d --name my-web-app -p 8080:80 nginxdemos/hello`.
- верифікація статусу виконання (Up time, зіставлення портів) через системний монітор `docker ps`.
- тестування транспортного та прикладного рівнів передачі даних за протоколом HTTP шляхом відправлення GET-запиту інструментом `curl http://localhost:8080`.
- аналіз зібраних логів доступу Nginx через перегляд стандартних потоків контейнера `docker logs my-web-app`.
- завершення активності процесу та очищення дискового шару контейнера послідовним викликом `docker stop my-web-app` та `docker rm my-web-app`.

Тестування програмної реалізації

Етап 1. Завантаження базового образу та верифікація локального реєстру

Мета: Перевірка зв'язку з публічним реєстром Docker Hub, завантаження необхідних шарів стисненого образу та підтвердження реєстрації образу в локальному списку.

У терміналі виконано запит на завантаження цільового образу:

```
docker pull nginxdemos/hello
```

Після отримання повідомлення про завершення вивантаження статусу Status: Downloaded newer image for nginxdemos/hello:latest, виконано перевірку реєстру локальних образів:

```
docker images
```

```
/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker pull nginxdemos/hello
Using default tag: latest
latest: Pulling from nginxdemos/hello
7a8a46741e18: Pull complete
8cd004bdedf3: Pull complete
c9ebe2ff2d2c: Pull complete
f5f620b01943: Pull complete
a992fbc61ecc: Pull complete
9824c27679d3: Pull complete
4c3bda1d1bf7: Pull complete
cb1ff4086f82: Pull complete
403e3f251637: Pull complete
5bc572a340ec: Pull complete
9adfbac99cb7: Pull complete
a83fb30b24b4: Download complete
5bfb5fcd3cf4: Download complete
Digest: sha256:b28d1e16c676896aaaa98ed9c36d406cc4f4289558487f15e37e04354720f807a
Status: Downloaded newer image for nginxdemos/hello:latest
docker.io/nginxdemos/hello:latest
/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
nginxdemos/hello:latest	b28d1e16c676	80.4MB	23.4MB	
rancher/klipper-helm:v0.11.1-build20260615	939b8f2fbb79	242MB	63.7MB	
rancher/klipper-lb:v0.4.17	910944bb0bd9	19.4MB	5.28MB	
rancher/local-path-provisioner:v0.0.36	1eba82e9c386	438MB	89.4MB	
rancher/mirrored-coredns-coredns:1.14.4	3e98f280fd60	104MB	23.7MB	
rancher/mirrored-library-busybox:1.37.0	101b4afd7673	6.55MB	2.15MB	
rancher/mirrored-library-traefik:3.7.4	fcdef599e625	243MB	53MB	
rancher/mirrored-metrics-server:v0.8.1	b2d2efaf5ac3	109MB	22.5MB	
rancher/mirrored-pause:3.6	74c4244427b7	986kB	298kB	

Рис. 1 — Завантаження базового образу

Аналіз результату: У терміналі успішно зафіксовано наявність репозиторію nginxdemos/hello з тегом latest. Система виділила локальний ідентифікатор образу та відобразила його реальний розмір на віртуальному диску, що підтверджує повну готовність образу до розгортання.

Етап 2. Ініціалізація та фоновий запуск контейнера з мапінгом портів

Мета: Запуск ізольованого процесу у фоновому режимі із налаштуванням маршрутизації мережевого трафіку через прокидання портів хоста.

Запуск виконано із явним зазначенням ідентифікатора та параметрів трансляції сокетів:

```
docker run -d --name my-web-app -p 8080:80 nginxdemos/hello
```

Для контролю працездатності процесу та перевірки мережевої конфігурації викликано команду:

```
docker ps
```

```
/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker run -d --name my-web-app
-p 8080:80 nginxdemos/hello
bd44d2f92c78d60c5d391daaf938572ffae055fc162606cc30742758c5dbef48
/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
bd44d2f92c78   nginxdemos/hello  "/docker-entrypoint..."  6 seconds ago  Up 6 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
my-web-app
```

Рис. 2 — Ініціалізація та фоновий запуск контейнера

Аналіз результату: Демон успішно згенерував повнорозмірний хеш контейнера та запустив його у фоновому режимі detached. Таблиця docker ps відображає створений процес під ім'ям my-web-app, демонструє статус Up, а також підтверджує коректний біндинг порту: 0.0.0.0:8080->80/tcp, що гарантує доступність внутрішнього веб-сервера з хостової машини.

Етап 3. Верифікація роботи веб-сервера та аудит системних логів

Мета: Підтвердження функціональної доступності додатка через HTTP-запит та дослідження системи журналювання веб-сервера всередині контейнера.

Для тестування відповіді веб-вузла надіслано HTTP GET-запит за адресою прокинутого порту:

```
curl http://localhost:8080
```

Для перегляду записів про вхідне мережеве з'єднання та стан ініціалізації Nginx отримано лог контейнера:

```
docker logs my-web-app
```

```

/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker logs my-web-app
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf is not a file or does not exist
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/03 14:54:48 [notice] 1#1: using the "epoll" event method
2026/10/03 14:54:48 [notice] 1#1: nginx/1.29.1
2026/10/03 14:54:48 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
2026/10/03 14:54:48 [notice] 1#1: OS: Linux 5.15.167.4-microsoft-standard-WSL2
2026/10/03 14:54:48 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/10/03 14:54:48 [notice] 1#1: start worker processes
2026/10/03 14:54:48 [notice] 1#1: start worker process 20
2026/10/03 14:54:48 [notice] 1#1: start worker process 21
2026/10/03 14:54:48 [notice] 1#1: start worker process 22
2026/10/03 14:54:48 [notice] 1#1: start worker process 23
2026/10/03 14:54:48 [notice] 1#1: start worker process 24
2026/10/03 14:54:48 [notice] 1#1: start worker process 25
2026/10/03 14:54:48 [notice] 1#1: start worker process 26
2026/10/03 14:54:48 [notice] 1#1: start worker process 27
2026/10/03 14:54:48 [notice] 1#1: start worker process 28
2026/10/03 14:54:48 [notice] 1#1: start worker process 29
2026/10/03 14:54:48 [notice] 1#1: start worker process 30
2026/10/03 14:54:48 [notice] 1#1: start worker process 31
172.17.0.1 - - [03/Oct/2026:14:54:59 +0000] "GET / HTTP/1.1" 200 12127 "-" "curl/8.21.0" "-"

```

Рис. 3 — Верифікація роботи веб-сервера та аудит системних логів

Аналіз результату: Веб-сервер повертає згенеровану HTML-сторінку Nginx із деталями конфігурації вузла (включаючи внутрішню IP-адресу та ім'я контейнера, що виступає хостнеймом). Журнал docker logs фіксує успішне підключення клієнта, код відповіді 200 та шлях /, що підтверджує коректність маршрутизації трафіку та безпомилкову роботу застосунку.

Етап 4. Завершення життєвого циклу контейнера та звільнення ресурсів

Мета: Демонстрація процедури безпечної зупинки процесу та остаточного видалення контейнера з системи.

Здійснено відправлення сигналу завершення та перевірено зміну статусу інстансу:

```
docker stop my-web-app
```

```
docker ps -a
```

Після підтвердження статусу Exited виконано остаточну утилізацію сутності:

```
docker rm my-web-app
```

```
docker ps -a
```

```

/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker stop my-web-app
my-web-app
/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
bd44d2f92c78   nginxdemos/hello "/docker-entrypoint...." 24 minutes ago Exited (0) 9 seconds ago          my-web-app
/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker rm my-web-app
my-web-app
/mnt/d/4_1/Cloud_Technologies/Lab_10 # docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES

```

Рис. 5 — Завершення життєвого циклу контейнера та звільнення ресурсів

Аналіз результату: Команда `docker stop` безпечно завершила основний процес Nginx, перевівши контейнер у стан Exited (0). Наступний виклик `docker rm` повністю вилучив зарезервованій шар запису файлової системи та очистив таблицю контейнерів, у чому можна переконатися за відсутністю `my-web-app` у фінальному лістингу `docker ps -a`.

Висновок

У ході виконання лабораторної роботи було досліджено теоретичні засади технології контейнеризації та успішно реалізовано повний життєвий цикл ізольованого програмного сервісу в середовищі Rancher Desktop.

Встановлено та сконфігуровано відкрите середовище виконання контейнерів на базі віртуальної платформи WSL2 із використанням контейнерного бекенду Moby (dockerd), що дало змогу використовувати стандартний інтерфейс командного рядка Docker CLI без застосування стороннього ліцензованого ПЗ.

У ході практичного тестування було верифіковано механізми взаємодії з публічним реєстром Docker Hub шляхом завантаження образу nginxdemos/hello. Реалізовано запуск веб-сервера у фоновому режимі із присвоєнням статичного системного імені my-web-app та мапінгом зовнішнього порту 8080 на внутрішній порт 80 контейнера. За допомогою утиліти curl підтверджено коректність функціонування механізмів трансляції портів та повернення HTTP-відповіді з кодом 200 OK. Було також відпрацьовано механізми моніторингу через аналіз стандартних потоків логування та продемонстровано завершальні стадії життєвого циклу контейнера (зупинка через сигнал завершення процесу та утилізація виділених системних ресурсів), що підтверджує повну відповідність поставленим вимогам.

Код програми

Код до даної лабораторної роботи не використовувався.